# Linux SID
# 开发指南

版本号: 3.4

发布日期: 2025-11-03

广州星翼电子科技有限公司zdyzlinux

# 版本历史

| 版本号 | 日期 | 制/修订人 | 内容描述 |
|---|---|---|---|
| 1.0 | 2020-07-19 | AWA1835 | 初始版本 |
| 2.0 | 2020-11-11 | AWA1538 | 适配 linux-5.4 平台。 |
| 2.1 | 2022-03-04 | AWA1832 | 更新函数说明。 |
| 2.2 | 2022-03-04 | AWA1832 | 此文档今后只支持内核版本为 Linux-5.4 及之后的平台。 |
| 2.3 | 2022-03-08 | AWA1768 | 修改 sid 的源码路径。 |
| 2.4 | 2023-11-25 | XAA0248 | 修改格式问题。 |
| 2.5 | 2024-07-15 | XAA0175 | 修改部分格式问题，优化有歧义的内容，增加章节 “动态调试接口”。 |
| 2.6 | 2024-10-18 | XAA0190 | 1. 增加第 6 章节 “FAQ”。 |
| 2.7 | 2024-12-09 | XAA0190 | 1. 修改第 6 章节 “FAQ” 格式。 |
| 2.8 | 2025-02-11 | XAA0190 | 1. 修改 2.3 章节，模块 device tree 配置内容。2. 修改第 3 章节，关键数据结构内容。3. 修改第 4 章节内部函数接口。 |
| 2.9 | 2025-03-26 | XAA0190 | 1. 修改 2.3 章节，模块 device tree 配置内容。 |
| 3.0 | 2025-07-26 | XAA0190 | 1. 修改 2.1 章节文档格式。 |
| 3.1 | 2025-08-06 | XAA0190 | 1. 修改第 2、3、4 章节内容。 |
| 3.2 | 2025-08-18 | XAA0190 | 1. 修改文档格式。 |
| 3.3 | 2025-08-26 | XAA0190 | 1. 更新模块使用 demo。 |
| 3.4 | 2025-11-03 | XAA0190 | 1. 修改文档格式。 |

版权所有 © 珠海全志科技股份有限公司。保留一切权利

# 目　录


**1 前言** **1**
1.1 编写目的 . . . . . 1
1.2 适用范围 . . . . . 1
1.3 目标读者 . . . . . 1
1.4 文档约定 . . . . . 1
1.4.1 标志说明 . . . . . 1
1.4.2 地址与数据描述方法约定 . . . . . 1
1.4.3 数值单位约定 . . . . . 2
1.5 术语、定义、缩略语 . . . . . 2

**2 模块描述** **3**
2.1 模块功能 . . . . . 3
2.1.1 Chip ID 功能 . . . . . 3
2.1.2 Efuse 功能 . . . . . 3
2.1.3 状态位 . . . . . 3
2.2 模块位置 . . . . . 4
2.3 模块 device tree 配置说明 . . . . . 4
2.4 模块源码结构 . . . . . 5
2.5 内核配置 . . . . . 5
2.5.1 SID 驱动 . . . . . 5
2.5.2 sysinfo 驱动 . . . . . 5

**3 模块设计** **7**
3.1 结构框图 . . . . . 7
3.2 关键数据定义 . . . . . 7
3.2.1 常量及宏定义 . . . . . 7
3.2.2 全局变量 . . . . . 8
3.3 模块流程设计 . . . . . 8
3.3.1 SoC 信息读取流程 . . . . . 8
3.3.2 Efuse Key 读取流程 . . . . . 8

**4 接口设计** **10**
4.1 驱动层对外接口 . . . . . 10
4.1.1 `int sunxi_get_soc_chipid(u8 *chipid)` . . . . . 10
4.1.2 `int sunxi_get_serial(u8 *serial)` . . . . . 10
4.1.3 `sunxi_get_soc_chipid_str(char *serial)` . . . . . 10
4.1.4 `int sunxi_get_soc_ft_zone_str(char *serial)` . . . . . 11
4.1.5 `int sunxi_get_soc_rotpk_status_str(char *status)` . . . . . 11
4.1.6 `int sunxi_soc_is_secure(void)` . . . . . 11


版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.1.7 unsigned int sunxi_get_soc_bin(void) . . . . . 11
4.1.8 s32 sunxi_efuse_readn(void *key_name, void *buf, u32 n) . . . 12
4.2 读 Efuse key 接口（驱动层） . . . . . 12

**5 可测试性** **13**
5.1 sysfs 调试接口 . . . . . 13
5.2 动态调试接口 . . . . . 13
5.3 应用层调用示例 . . . . . 13

**6 应用层读写 Efuse 示例** **15**
6.1 demo 接口介绍 . . . . . 19
6.1.1 sunxi_efuse_oem_read . . . . . 19
6.1.2 sunxi_efuse_oem_read_with_offset . . . . . 19
6.1.3 sunxi_efuse_oem_write . . . . . 20
6.1.4 sunxi_efuse_oem_write_with_offset . . . . . 20
6.1.5 sunxi_efuse_chipid_read . . . . . 20
6.1.6 sunxi_efuse_chipid_read_with_offset . . . . . 21

**7 FAQ** **22**

广州星翼电子科技有限公司zdyzlinux

# 1 前言

## 1.1 编写目的

介绍 Linux 内核中基于 Sunxi 硬件平台的 SID 模块驱动的详细设计，为软件编码和维护提供基础。

## 1.2 适用范围

内核版本为 Linux-5.4 及之后的平台。

## 1.3 目标读者

SID 驱动、Efuse 驱动、Sysinfo 驱动的维护或应用开发人员等。

## 1.4 文档约定

### 1.4.1 标志说明

警告

- 提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。

注意

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

### 1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

表 1-1: 地址与数据描述方法约定

| 符号 | 例子 | 说明 |
|---|---|---|
| 0x | 0x0200，0x79 | 地址或数据以 16 进制表示。 |
| 0b | 0b010，0b00 000 111 | 数据采用二进制表示 (寄存器描述除外)。 |
| X | 00X，XX1 | 数据描述中，X 代表 0 或 1。 |
| | | 例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。 |

### 1.4.3 数值单位约定

本文档在描述数据容量时，若无显示说明单位，则 “对应数值” 以 bit 为单位，文档中各单位与bit的关系如下表：

表 1-2: 数值单位约定

| 类型 | 符号 | 对应数值 |
|---|---|---|
| 数据容量 | 1 bit | 1 bit |
| | 1 Byte | 8 bit |
| | 1 Word | 128 bit |

## 1.5 术语、定义、缩略语

表 1-3: 术语表

| 术语或缩略 | 描述 |
|---|---|
| Efuse | Electronic Fuse，电子熔丝。 |
| SID | Security ID，特指 AW SoC 中的 SID 模块。 |
| Sysinfo | 一个字符型设备驱动，用于方便用户空间获取、调试 SID 信息。 |
| sunxi | 指 Allwinner 的一系列 SOC 硬件平台。 |

版权所有 © 珠海全志科技股份有限公司。保留一切权利

# 2 模块描述

## 2.1 模块功能

SID 提供的功能可以分为 3 个部分：ChipID、Efuse 功能、以及一些状态位。

### 2.1.1 Chip ID 功能

对于全志的 SoC 来说，`ChipID`用于该 SoC 的唯一标识，如`A*`的`ChipID`用于标识任一颗`A* SoC`在所有的`A*`中是唯一的（目前仅保证同一型号 SoC 中的`ChipID`唯一）。

`ChipID`由 4 个 word（16 Byte）组成，共 128 bit。

### 2.1.2 Efuse 功能

对软件来说，Efuse 中提供了一个可编程的永久存储空间，特点是每一位只能写一次（从 0 写为 1）。

Efuse 接口方式：Efuse 容量大于 512 bit 采用 SRAM 方式。带有 SRAM 的硬件结构示意图如下：

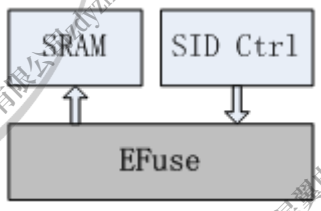


图 2-1: 带有 SRAM 的 SID 模块硬件结构

### 2.1.3 状态位

Secure Enable 标志位。

1. 标明当前系统的 Security 属性是否打开，即是否运行了 SecureBoot 和 SecureOS。
2. 芯片 SecureEnable 状态位通常保存在 SID 模块的`0xa0`寄存器（Secure Mode Status Register）。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

## 2.2 模块位置

SID 是一个比较独立的模块，在 Linux 内核中没有依赖其他子系统，在 Sunxi 平台存放在`{SDK}/bsp/drivers/sid/`目录中。SID 为其他模块提供 API 的调用方式关系图如下：

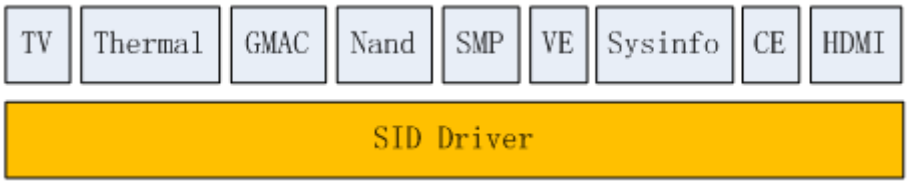


图 2-2: SID 与其他模块的关系

- TV、Thermal、GMAC 的校准参数保存在 SID 中。
- Nand、SMP、VE 需要读取 SoC Version。
- CE 和 HDMI 会用到 SID 中的一些 Key。
- Sysinfo 比较特殊，它是为了方便用户空间获取、调试 SID 信息，而专门设计的一个字符型设备驱动。

## 2.3 模块 device tree 配置说明

注意

*dtsi* 文件一般无需改动直接使用即可。

在 soc 级的 **dtsi** 文件中配置了寄存器基地址等共性信息。soc 级的 **dtsi** 文件的路径为：`${SDK}/bsp/configs/{linux-ver}/{CHIP}.dtsi`，其中`{CHIP}`为研发代号，如`sun50iw10p1`。

以`sun50iw10p1`为例，需要在`sun50iw10p1.dtsi`中包含如下配置：

```
sid@3006000 {
        compatible = "allwinner,sun50iw10p1-sid", "allwinner,sunxi-sid";
        reg = <0x0 0x03006000 0 0x1000>;
        #address-cells = <1>;
        #size-cells = <1>;
        non-secure-maxoffset = <0x80>;
        non-secure-maxlen = <0x20>;

        secure_status {
            reg = <0x0 0>;
            offset = <0xa0>;
            size = <0x4>;
        };
        chipid {
            reg = <0x0 0>;
            offset = <0x200>;
            size = <0x10>;
        };
        rotpk {
            reg = <0x0 0>;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
            offset = <0x140>;
            size = <0x20>;
        };
    };
```

在dtsi文件sid节点下增加子节点secure_status，chipid，rotpk，就可以用 sysfs 节点key_info来访问：

```
console:/ # echo secure_status > /sys/class/sunxi_info/key_info; cat /sys/class/sunxi_info/
    key_info
console:/ # 1
```

## 2.4 模块源码结构

SID 驱动的源码如下：

```
${SDK}/bsp/
├── include/
│   └── sunxi-sid.h
├── drivers/smc/
│   └── sunxi-sysinfo.c
└── drivers/sid/
    ├── Kconfig
    ├── Makefile
    └── sunxi-sid.c
```

## 2.5 内核配置

### 2.5.1 SID 驱动

SID 驱动的内核配置项为CONFIG_AW_SID。此配置项一般默认打开，不需要重新配置。

若要配置，可在 SDK 根目录执行./build.sh menuconfig：

```
Allwinner BSP
└──> Device Drivers
    └──> SID Drivers
        └──> Allwinner SID Support
```

### 2.5.2 sysinfo 驱动

若要在用户空间通过 sysfs 访问 Efuse 里部分信息，则需要打开 sysinfo 驱动。对应的配置项为CONFIG_AW_SYS_INFO，驱动代码为drivers/smc/sunxi-sysinfo.c。

若要配置，可在 SDK 根目录执行./build.sh menuconfig：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
Allwinner BSP
└──> Device Drivers
    └──> SMC Drivers
        └──> Allwinner System Info Driver
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

# 3 模块设计

## 3.1 结构框图

SID 驱动内部的功能划分如下图所示：

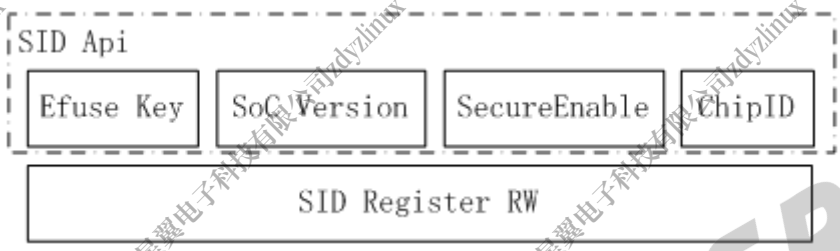


图 3-1: SID 驱动的内部结构

总体上，SID 驱动内部可以分为两大部分：

1. SID Register RW：封装了对寄存器按位读取的接口，以及获取指定 compatible 的模块基地址等。
2. SID API：以 API 的方式提供一些功能接口，获取 Key、获取 SecureEnable、获取 ChipID 等。

## 3.2 关键数据定义

### 3.2.1 常量及宏定义

- key 的名称定义

在获取 Key 的时候，调用者需要知道 Key 的名称，以下述作为索引的依据。

```
#define EFUSE_CHIPID_NAME           "chipid"
#define EFUSE_OEM_NAME              "oem"
#define EFUSE_ROTPK_NAME            "rotpk"
#define EFUSE_SSK_NAME              "ssk"
#define EFUSE_HDCP_HASH_NAME        "hdcphash"
#define EFUSE_HDCP_PKF_NAME         "hdcppkf"
#define EFUSE_NV1_NAME              "nv1"
#define EFUSE_NV2_NAME              "nv2"
#define EFUSE_RESERVED_NAME         "reserved"
#define EFUSE_OEM_SECURE_NAME       "oem_secure"
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

注意

并非上述所有 *key* 都能访问，一般在dtsi中定义的可以直接访问。

### 3.2.2 全局变量

定义几个 static 全局变量，用于保存解析后的 ChipID 等信息。

```
static unsigned int sunxi_soc_chipid[4];
static unsigned int sunxi_serial[4];
static int sunxi_soc_secure;
static unsigned int sunxi_soc_bin;
```

## 3.3 模块流程设计

### 3.3.1 SoC 信息读取流程

本节中，这里把 ChipID、SecureEnable 信息统称为 “SoC 信息”，因为他们的读取过程非常相似，都是遵循以下流程：

图 3-2: SoC 信息读取流程

### 3.3.2 Efuse Key 读取流程

在读取 Efuse 中 Key 的时候，需要判断 key 是否存在、以及 key 访问权限，过程稍微复杂些，用以下流程图进行简单说明：

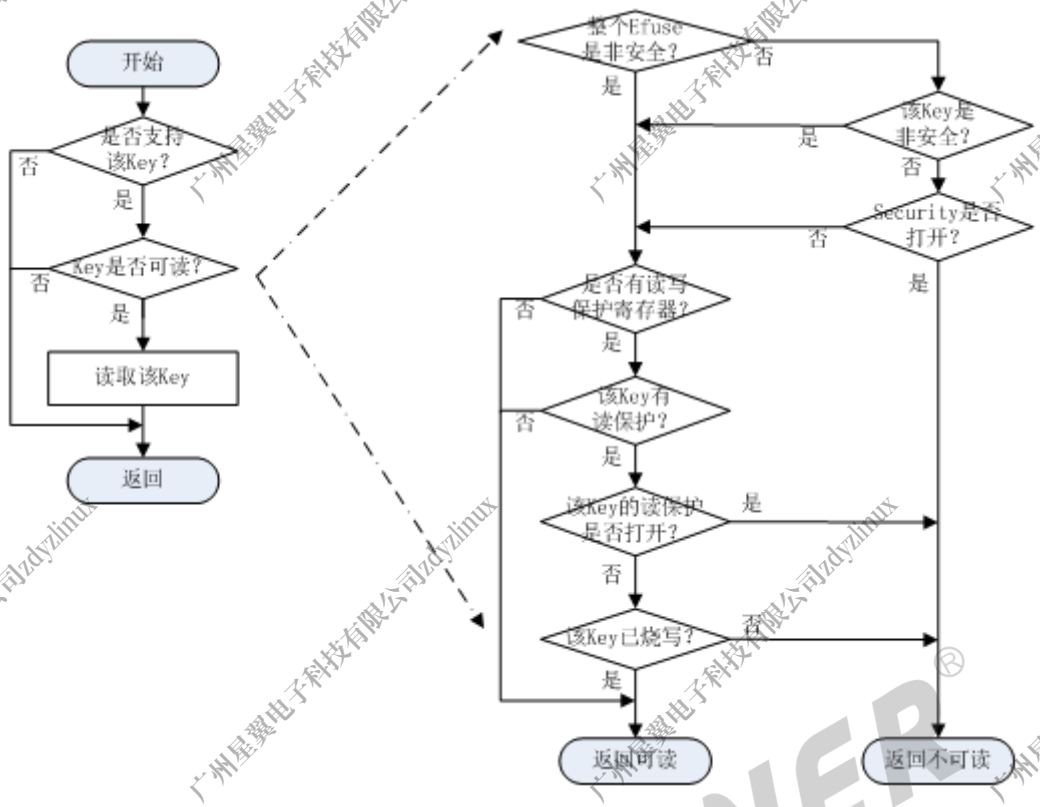


图 3-3: Efuse Key 的读取流程

广州星翼电子科技有限公司zdyzlinux

# 4 接口设计

## 4.1 驱动层对外接口

### 4.1.1 int sunxi_get_soc_chipid(u8 *chipid)

- 作用：获取 SoC 的 ChipID（从 Efuse 中读到的原始内容，包括数据内容和顺序）。
- 参数：
  - chipid：用于保存 ChipID 的缓冲区。
- 返回：
  - 会返回0，无实际意义。

### 4.1.2 int sunxi_get_serial(u8 *serial)

- 作用：获取 SoC 的序列号（由 ChipID 加工而来）。
- 参数：
  - serial：用于保存序列号的缓冲区。
- 返回：
  - 会返回0，无实际意义。

### 4.1.3 sunxi_get_soc_chipid_str(char *serial)

- 作用：获取 SoC 的 ChipID 的第一个字节，要求转换为字符串格式。
- 参数：
  - serial：用于打印 ChipID 第一个字节的缓冲区。
- 返回：
  - 只会返回8（4 个字节的十六进制打印长度），无实际意义。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

### 4.1.4 int sunxi_get_soc_ft_zone_str(char *serial)

- 作用：获取 FZ ZONE 的最后一个字节，要求转换为字符串格式。
- 参数：
  - `serial`：用于打印 ChipID 第一个字节的缓冲区。
- 返回：
  - 只会返回`8`（4 个字节的十六进制打印长度），无实际意义。

### 4.1.5 int sunxi_get_soc_rotpk_status_str(char *status)

- 作用：获取 rotpk 的状态，是否烧码。
- 参数：
  - `status`：用于记录是否烧码的缓冲区；`0`，未烧；`1`，已烧。
- 返回：
  - `%d`的长度，无实际意义。

### 4.1.6 int sunxi_soc_is_secure(void)

- 作用：获取整个系统的 Secure 状态，即安全系统是否启用。
- 参数：
  - 无。
- 返回：
  - `0`，未启用安全系统；`1`，启用。

### 4.1.7 unsigned int sunxi_get_soc_bin(void)

- 作用：用于芯片分 bin，部分 SoC 平台才支持。
- 参数：
  - 无。
- 返回：
  - `0`: fail。
  - `1`: normal。
  - `2`: faster。
  - `3`: fastest。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

### 4.1.8 s32 sunxi_efuse_readn(void *key_name, void *buf, u32 n)

- 作用：读取 Efuse 中的一个 key 信息。
- 参数：
  - `key_name` - Key 的名称，定义详见`sunxi-sid.h`。
  - `buf` - 用于保存 Key 值的缓冲区。
  - `size` - buf 的大小。
- 返回：
  - `0`: success。
  - other: fail。

## 4.2 读 Efuse key 接口（驱动层）

```
s32 sunxi_efuse_readn(void *key_name, void *buf, u32 n)
```

- 作用：读取 Efuse 中的一个 key 信息。
- 参数：
  - `key_name`: Key 的名称，定义详见`sunxi-sid.h`。
  - `buf`: 用于保存 Key 值的缓冲区。
  - `size`: buf 的大小。
- 返回：
  - `0`: success。
  - other: fail。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

# 5 可测试性

## 5.1 sysfs 调试接口

1. /sys/class/sunxi_info/sys_info 此节点文件可以打印出一些 SoC 信息，包括版本信息、ChipID 等。

```
# cat /sys/class/sunxi_info/sys_info
sunxi_platform     : T153
sunxi_secure       : secure    // 当前系统是否安全固件
sunxi_rotpk        : 0         // 当前系统是否烧写rotpk
...
```

2. /sys/class/sunxi_info/key_info 此节点用于获取指定名称的 Key 信息，方法是先写入一个 Key 名称，然后就可以读取到 Key 的内容。

```
# echo chipid > /sys/class/sunxi_info/key_info
# cat /sys/class/sunxi_info/key_info
22800000 48004908 00c7205c 089b1d8b
```

## 5.2 动态调试接口

SID 驱动使用动态调试接口（比如pr_debug()）来打印部分调试信息。若要查看这部分打印信息，请按如下步骤执行：

1. 打开内核配置中的CONFIG_DYNAMIC_DEBUG宏。
2. 在小机端，使能指定的 debug 信息：

```
# 挂载debugfs：
mount -t debugfs none /sys/kernel/debug/

# 打开指定文件的动态debug（+p表示打开，-p表示关闭）：
echo "file sunxi-sid.c +p" > /sys/kernel/debug/dynamic_debug/control

# 打开指定行的动态debug：
echo "file sunxi-sid.c line 154 +p" > /sys/kernel/debug/dynamic_debug/control
```

## 5.3 应用层调用示例

应用层可通过设备节点/dev/sunxi_soc_info读取部分 Efuse 信息，具体使用可参考下述 demo。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

表 5-1: ioctl 命令含义表

| cmd | 含义 |
| --- | --- |
| CHECK_SOC_CHIPID | read chipid informations |
| CHECK_SOC_CHIPID_ORIGIN | read origin chipid informations |
| CHECK_SOC_FT_ZONE | read ft_zone informations |

```
enum {
        CHECK_SOC_CHIPID = 4,
        CHECK_SOC_FT_ZONE = 5,
        CHECK_SOC_CHIPID_ORIGIN = 8,
};

// 打开设备节点

fd = open("/dev/sunxi_soc_info",O_RDONLY);
if (fd < 0){
    printf("open /dev/sunxi_soc_info failed!");
    return -1;
}

// 通过ioctl读取Efuse信息
ret = ioctl(fd,cmd,buf);

if (ret < 0) {
    printf("ioctl err!\n");
    close(fd);
    return ret;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

# 6 应用层读写 Efuse 示例

应用层可通过设备节点/dev/sid_efuse读取 Efuse 中key信息，具体使用可参考下述 demo：

注意

下述读 *key* 接口仅可读取在*.dtsi文件中配置的 *key*

具体 demo 示例如下：

```
#define EFUSE_READ _IO('V', 1)
#define EFUSE_WRITE _IO('V', 2)
#define EFUSE_IOCTL_PATH ("/dev/sid_efuse")

struct sunxi_efuse_key_info {
        char name[64];
        uint32_t len;
        uint32_t offset;
        uint64_t key_data;
};

/* efuse 对/dev/sid_efuse节点操作（读/写）接口 */
static int sunxi_efuse_ioctl(struct sunxi_efuse_key_info *key_info, int type)
{
        int fd, ret = -1;

        if (!key_info) {
                printf("Failed to call efuse ioctl: the key_info is NULL\n");
                return -1;
        }

        fd = open(EFUSE_DEVICE_PATH, O_RDONLY);
        if (fd < 0) {
                printf("Failed to open '%s': '%s'\n", EFUSE_DEVICE_PATH, strerror(errno));
                return -1;
        }

        switch (type) {
        case EFUSE_READ:
                ret = ioctl(fd, EFUSE_READ, key_info);
                if (ret < 0)
                        printf("Failed to read efuse, error num = %d '%s'\n", ret, strerror(
    errno));
                break;
        case EFUSE_WRITE:
                ret = ioctl(fd, EFUSE_WRITE, key_info);
                if (ret < 0)
                        printf("Failed to write efuse, error num = %d '%s'\n", ret, strerror(
    errno));
                break;
        default:
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
                printf("Unsupport ioctl type %d\n", type);
                ret = -1;
                break;
        };

        close(fd);
        return ret;
}

/* 读efuse接口 */
static int sunxi_efuse_read(char *key_name, uint8_t *buf, uint32_t len, uint32_t offset)
{
        struct sunxi_efuse_key_info *key_info;
        int ret = 0;
        uint8_t *key_buf;

        if (!key_name) {
                printf("The key_name is NULL\n");
                return -1;
        }

        if (len + offset > KEY_MAX_SIZE) {
                printf("The key len + offset > KEY_MAX_SIZE, please check\n");
                return -1;
        }

        key_buf = malloc(KEY_MAX_SIZE);
        if(!key_buf) {
                printf("Failed to malloc key_buf\n");
                return -1;
        }
        memset(key_buf, 0, KEY_MAX_SIZE);

        key_info = malloc(sizeof(*key_info));
        if(!key_info) {
                printf("Failed to malloc key_info\n");
                ret = -1;
                goto err0;
        }
        memset(key_info, 0, sizeof(*key_info));

        memcpy(key_info->name, key_name, strlen(key_name));
        key_info->key_data = (uint64_t)(unsigned long)key_buf;
        key_info->offset = offset;
        key_info->len = offset + len;

        ret = sunxi_efuse_ioctl(key_info, EFUSE_READ);
        if (ret) {
                printf("Failed to read key '%s'\n", key_name);
                ret = -1;
                goto err1;
        }
        memcpy(buf, key_buf + offset, len);

err1:
        free(key_info);
err0:
        free(key_buf);

        return ret;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
}

/* 写efuse接口 */
static int sunxi_efuse_write(char *key_name, const uint8_t *buf, uint32_t len, uint32_t
    offset)
{
        struct sunxi_efuse_key_info *key_info;
        char *key_buf = NULL;
        int ret = 0;

        if (!key_name) {
                printf("The key_name is NULL\n");
                return -1;
        }

        key_info = malloc(sizeof(*key_info));
        if(!key_info) {
                printf("Failed to malloc key_info\n");
                return -1;
        }
        memset(key_info, 0, sizeof(*key_info));

        key_buf = malloc(KEY_MAX_SIZE);
        if (!key_buf) {
                printf("Failed to malloc key_buf\n");
                goto err0;
        }
        memset(key_buf, 0, KEY_MAX_SIZE);
        memcpy(key_buf, buf, len);

        memcpy(key_info->name, key_name, strlen(key_name));
        key_info->key_data = (uint64_t)(unsigned long)key_buf;
        key_info->offset = offset;
        key_info->len  = len;

        ret = sunxi_efuse_ioctl(key_info, EFUSE_WRITE);
        if (ret) {
                printf("Failed to write key '%s'\n", key_name);
                ret = -1;
                goto err1;
        }

err1:
        free(key_buf);
err0:
        free(key_info);

        return ret;
}

/* 读oem key接口，从oem key内偏移offset处开始读 */
int sunxi_efuse_oem_read_with_offset(uint8_t *buf, uint32_t offset, uint32_t len)
{
        char key_name[18] = "oem";
        int ret = 0;

        if (!buf) {
                printf("The key 'oem' buf is NULL, please check\n");
                return -1;
        }
```

```
        if (!len) {
                printf("The key 'oem' len is 0, please check\n");
                return -1;
        }

        ret = sunxi_efuse_read(key_name, buf, len, offset);

        return ret;
}

/* 读oem key接口 */
int sunxi_efuse_oem_read(uint8_t *buf, uint32_t len)
{
        return sunxi_efuse_oem_read_with_offset(buf, 0, len);
}

/* 写oem key接口，从oem key内偏移offset处开始写 */
int sunxi_efuse_oem_write_with_offset(const uint8_t *buf, uint32_t offset, uint32_t len)
{
        char key_name[4] = "oem";
        int ret = 0;

        if (!buf) {
                printf("The key 'oem' buf is NULL, please check\n");
                return -1;
        }

        if (!len) {
                printf("The key 'oem' len is 0, please check\n");
                return -1;
        }

        ret = sunxi_efuse_write(key_name, buf, len, offset);

        return ret;
}

/* 写oem key接口 */
int sunxi_efuse_oem_write(const uint8_t *buf, uint32_t len)
{
        return sunxi_efuse_oem_write_with_offset(buf, 0, len);
}

/* 读chipid key接口，从chipid key内偏移offset处开始读 */
int sunxi_efuse_chipid_read_with_offset(uint8_t *buf, uint32_t offset, uint32_t len)
{
        char key_name[7] = "chipid";
        int ret = 0;

        if (!buf) {
                printf("The key 'chipid' buf is NULL, please check\n");
                return -1;
        }

        if (!len) {
                printf("The key 'chipid' len is 0, please check\n");
                return -1;
        }
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        ret = sunxi_efuse_read(key_name, buf, len, offset);

        return ret;
}

/* 读chipid key接口 */
int sunxi_efuse_chipid_read(uint8_t *buf, uint32_t len)
{
        return sunxi_efuse_chipid_read_with_offset(buf, 0, len);
}
```

## 6.1 demo 接口介绍

上述述 demo 中已提供oem 与 chipid的操作接口，可直接参考使用

```
int sunxi_efuse_oem_read(uint8_t *buf, uint32_t len);
int sunxi_efuse_oem_read_with_offset(uint8_t *buf, uint32_t offset, uint32_t len);
int sunxi_efuse_oem_write(const uint8_t *buf, uint32_t len);
int sunxi_efuse_oem_write_with_offset(const uint8_t *buf, uint32_t offset, uint32_t len);
int sunxi_efuse_chipid_read(uint8_t *buf, uint32_t len);
int sunxi_efuse_chipid_read_with_offset(uint8_t *buf, uint32_t offset, uint32_t len);
```

接口具体介绍如下。

### 6.1.1 sunxi_efuse_oem_read

- 函数原型:`int sunxi_efuse_oem_read(char *buf, uint32_t len);`。
- 函数功能: 读取oem中长度为len的数据。
- 函数参数:
  - buf: 保存读出的数据;
  - len: 读取的长度（单位:byte）。
- 函数返回值:
  - 0: 成功;
  - 其他: 失败。

### 6.1.2 sunxi_efuse_oem_read_with_offset

- 函数原型:`int sunxi_efuse_oem_read_with_offset(char *buf, uint32_t offset, uint32_t len);`。
- 函数功能: 读取oem中数据，从oembase + offset开始读取, 长度为len。
- 函数参数:
  - buf: 保存读出的数据;

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - len: 读取的长度 (单位：byte)；
  - offset: 基于 oem 地址的偏移 (单位：byte)。
- 函数返回值:
  - 0: 成功;
  - 其他: 失败。

### 6.1.3 sunxi_efuse_oem_write

- 函数原型:int sunxi_efuse_oem_write(char *buf, uint32_t len);。
- 函数功能: 烧写oem，长度为len。
- 函数参数:
  - buf: 待烧写数据;
  - len: 烧写的数据长度 (单位：byte)。
- 函数返回值:
  - 0: 成功;
  - 其他: 失败。

### 6.1.4 sunxi_efuse_oem_write_with_offset

- 函数原型:int sunxi_efuse_oem_write_with_offset(char *buf, uint32_t offset, uint32_t len);。
- 函数功能: 烧写oem, 从oem_base + offset地址开始烧写，长度为len。
- 函数参数:
  - buf: 保存待写入数据;
  - len: 烧写的数据长度（单位：byte）;
  - offset: 烧写的偏移地址（基于oem地址的偏移, 单位：byte）。
- 函数返回值:
  - 0: 成功;
  - 其他: 失败。

### 6.1.5 sunxi_efuse_chipid_read

- 函数原型:int sunxi_efuse_chipid_read(char *buf, uint32_t len);。
- 函数功能: 读取chipid数据，长度为len。
- 函数参数:
  - buf: 保存读取的数据;

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - len: 读取的数据长度（单位byte）。

- 函数返回值:
  - 0: 成功;
  - 其他: 失败。

### 6.1.6 sunxi_efuse_chipid_read_with_offset

- 函数原型:`int sunxi_efuse_chipid_read_with_offset(char *buf, uint32_t offset, uint32_t len);`。
- 函数功能: 读取chipid数据，从chipid_base + offset开始读取, 长度为len。
- 函数参数:
  - buf: 保存读取的数据;
  - len: 读取的长度（单位：byte）；
  - offset: 基于chipid基地址的偏移(单位：byte)。
- 函数返回值:
  - 0: 成功;
  - 其他: 失败。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

# 7 FAQ

1. 若系统无法访问 Efuse 信息，应如何处理?

答案：针对部分产品，当启用安全系统后，Non-Secure 空间将无法访问大部分的 Efuse 信息，这个时候需要通过 SMC 指令来读取这些 Key 信息。此时不能再使用普通的寄存器读接口，而需要调用 SMC 接口：`int arm_svc_efuse_read(phys_addr_t key_buf, phys_addr_t read_buf)`。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

**著作权声明**

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

**商标声明**

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

**免责声明**

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。